

Course: Image Processing 31651

Assignment #24

Pixel to Pixel Operations (Part 4)

1

	ID (4 last digits)	Shorten Name	Photo of the student	
Student #1	1950	shienfeld		
Student #2	2210	pony		
Student #3	7939	akimov		

1

Assignment #24: Count Objects

In the file ImProcInPlainC.h defined numerical values of:
NUMBER_OF_ROWS and NUMBER_OF_COLUMNS

#	Image Description
1	Create synthetic image (Image241.bmp) that will have 6 Gaussians positioned in the pseudo-random way on the gray background. It is forbidden to create significant jumps in the gray levels of the resulting image Hint: use something like was used in the ADD RECTANGLE but by using INTEGER of FLOAT pixels
2	Find PHOTO (not a computer-created image) containing (5..20) objects of the (approximately) same size and (approximately) the same color with non-trivial background Convert the photo to Gray Image (Image242.bmp)
3	Create ALGORITHM that will COUNT "objects" The algorithm must use pointers to access the pixels.
4	Run Algorithm for Image241.bmp and for Image242.bmp and explain results
	For report use YOUR OWN STYLE. From YOUR Power Point customer must understand what was done, how and what are the results (Lecturer may later ask to add slides) Done: 24.1 , 24.2 24.3 24.4 REMOVE ALL RED when the items are ready Important hint: Fill report pages while working. Do not complain "not enough time" in case you did not follow THIS rule.

Pay attention : due to the complexity of the code and the complexity of the order of operations in the code – the functions will be explained first and only after that the connectivity to the task will be explained. Every function will be explained as an “isolated part” of the entire program.

24.1 – “DrawGaussian” function

3

The DrawGaussian function is designed to add a Gaussian distribution to an image at specified coordinates.

```
53  /* Adds a Gaussian distribution to the image at the given coordinates */
54  void DrawGaussian(unsigned char img[][NUMBER_OF_COLUMNS], int centerX, int centerY, float sigmaX, float sigmaY)
55  {
56      double a, b, c;
57      unsigned char d;
58      for (int row = 0; row < NUMBER_OF_ROWS; row++)
59      {
60          for (int column = 0; column < NUMBER_OF_COLUMNS; column++)
61          {
62              a = (column - centerX) / sigmaX;
63              b = (row - centerY) / sigmaY;
64              c = 255.0 * exp(-a * a - b * b);
65              d = (unsigned char)(int)(c + 0.5);
66              img[row][column] = (d > 10) ? d : img[row][column];
67          }
68      }
69  }
```

For each pixel located at (row, column), the function calculates the value of the Gaussian distribution at that point.

a and b are the normalized distances from the center of the Gaussian in the x and y directions, respectively.

The Gaussian value c is then calculated using the formula for a 2D Gaussian distribution:

d is the Gaussian value rounded to the nearest integer.

The function updates the pixel value at (row, column) in the image array img: If the calculated Gaussian value d is greater than 10, the pixel value is updated to d. If d is less than or equal to 10, the pixel value remains unchanged.

24.2 – “ROI (region of interest)” struct

4

```
22 // Struct for defining a region of interest (ROI)
23 struct ROI
24 {
25     int top;
26     int bottom;
27     int left;
28     int right;
29
30     // Method to clip the ROI to the bounds of the image
31     void ClipToBounds(int maxRows, int maxCols)
32     {
33         // Clip the ROI's top and bottom to [0, maxRows)
34         if (top < 0) top = 0;
35         if (bottom > maxRows) bottom = maxRows;
36
37         // Clip the ROI's left and right to [0, maxCols)
38         if (left < 0) left = 0;
39         if (right > maxCols) right = maxCols;
40
41         // Ensure ROI is valid (top < bottom and left < right)
42         if (top >= bottom) bottom = top;
43         if (left >= right) right = left;
44     }
45 };
```

The primary purpose of the ROI struct is to define a rectangular area within an image where certain operations will be applied.



This allows focusing on specific parts of the image rather than processing the entire image.



void **ClipToBounds** is a method to ensure the ROI is within the bounds of the image. It adjusts the top, bottom, left, and right properties to make sure they do not exceed the image dimensions.

Summary : we created the ROI as a useful tool for later counting operation (of objects or gaussians). In the next slides the usefulness of the ROI will be clarified!

24.3 – “ContElemInPicture” function

This function is designed to count elements in a specified region of interest (ROI) in a grayscale image and to mark these elements and the background.

```
93 // Count elements in the image based on the ROI
94 int ContElemInPicture(unsigned char GrayImage[][NUMBER_OF_COLUMNS], ROI MyROI)
95 {
96     int elementCount = 0; // Variable to count elements
97     unsigned char* pixelPtr;
98
99     for (int row = MyROI.top; row < MyROI.bottom; row++)
100     {
101         pixelPtr = GrayImage[row] + MyROI.left;
102         for (int col = MyROI.left; col < MyROI.right; col++, pixelPtr++)
103         {
104             if (*pixelPtr < 127)
105             {
106                 elementCount++;
107                 *pixelPtr = 50; // Mark the element
108             }
109             else
110             {
111                 *pixelPtr = 200; // Mark as background
112             }
113         }
114     }
115
116     return elementCount;
117 }
118
```

The function counts the number of elements (pixels) within a specified ROI in a grayscale image where the pixel values are below a certain threshold (127 in this case).

It also marks the pixels that are considered elements and those that are considered background.

Pay attention : **pixelPtr** is declared as a pointer to traverse the pixel values in the image. All the operations will be applied on that pointer.

24.3 – “InitThresholdLUTpart1” function

The InitThresholdLUTpart1 function initializes a Lookup Table for thresholding a grayscale image.



The LUT is used to map the pixel values based on a specified threshold, either making them black or white depending on whether they are above or below the threshold.

```
47 void InitThresholdLUTpart1(unsigned char* LUT, unsigned char Threshold, int B_Or_W)
48 {
49     for (int i = 0; i < myMAXCOLORS; i++)
50         LUT[i] = (255 - B_Or_W * 255) * (i > Threshold) + 255 * (i <= Threshold && B_Or_W);
51 }
```

Pay attention – one of the input of the function is a pointer to the LUT which was defined as global.

B_Or_W is a flag that determines the output value for pixels based on the threshold



If B_Or_W is 1 → pixels less than or equal to the threshold are set to 255 (white), and pixels greater than the threshold are set to 0 (black).

If B_Or_W is 0, pixels less than or equal to the threshold are set to 0 (black), and pixels greater than the threshold are set to 255 (white).

24.4 – “InitThresholdLUT” function

The InitThresholdLUT function initializes a Lookup Table for thresholding a grayscale image.

The LUT maps pixel values to either black (0) or white (255) **based on a specified range of values** – this is the difference from the other function InitThresholdLUTpart1.

Pixels within the range are set to white, and pixels outside the range are set to black.

Pay attention:

We thought that creating two functions for initializing LUT is the preferred way to approach the two very different tasks within that assignment. Although we will always prefer to write the most generic function that possible – here we decided to do different

```
71 // Initialize a lookup table (LUT) for thresholding
72 void InitThresholdLUT(unsigned char* LUT, unsigned char minVal, unsigned char maxVal)
73 {
74     for (int i = 0; i < myMAXCOLORS; i++)
75     {
76         // If pixel value is within the range [minVal, maxVal], set to white (255)
77         // Else, set to black (0)
78         if (i >= minVal && i <= maxVal)
79             LUT[i] = 255; // White
80         else
81             LUT[i] = 0;    // Black
82     }
83 }
```

The function is used to create **a binary thresholding effect** on a grayscale image. By converting pixel values within a specified range to white and those outside the range to black, we can separate objects or features of interest from the background.

24.5 – “ImposeLUT” function

After defining the LUT – we need to apply the LUT of the original gray image.

```
85 // Apply the LUT to the grayscale image
86 void ImposeLUT(unsigned char GrayImage[][NUMBER_OF_COLUMNS], unsigned char* LUT)
87 {
88     unsigned char* ptrToPixels = GrayImage[0];
89     for (int pixel = 0; pixel < NUMBER_OF_ROWS * NUMBER_OF_COLUMNS; pixel++)
90         *ptrToPixels++ = LUT[*ptrToPixels];
91 }
92
```

ptrToPixels is initialized to point to the first pixel of the image. Since GrayImage is a 2D array, GrayImage[0] points to the first row, and ptrToPixels thus points to the first pixel in that row.

24.6 – “main” function – part 1

9

This part of the “main” sets up the necessary variables and data structures, initializes random number generation, and prepares the images for further processing steps that will follow in the rest of the main function.

```
124 void main()
125 {
126     ROI MyROI;
127     int X, Y, BPixelCountElem = 0, BPixelCount = 0;
128
129     // Define ROI for the whole image
130     MyROI.top = 0;
131     MyROI.bottom = NUMBER_OF_ROWS;
132     MyROI.left = 0;
133     MyROI.right = NUMBER_OF_COLUMNS;
134
135     // Initialize random number generator with current time
136     std::mt19937 rng(static_cast<unsigned int>(std::chrono::system_clock::now().time_since_epoch().count()));
137
138     // Create uniform distribution for X and Y
139     std::uniform_int_distribution<int> distX(0, NUMBER_OF_COLUMNS - 1);
140     std::uniform_int_distribution<int> distY(0, NUMBER_OF_ROWS - 1);
141
142     //part1
143     // Initialize both images to black
144     for (int row = 0; row < NUMBER_OF_ROWS; row++)
145     {
146         for (int col = 0; col < NUMBER_OF_COLUMNS; col++)
147         {
148             ProccesIMG[row][col] = 0;
149             gaussian[row][col] = 0;
150         }
151     }
```

That line's purpose is to create a seeded random number generator for generating random coordinates or values later in the code.

These line's purpose is To define distributions for generating random coordinates within the image dimensions.

This double loop initializes all pixels in both ProccesIMG and gaussian images to black.

24.6 – “main” function – part 2

10

This part performs image processing operations to draw Gaussian distributions, apply thresholding using LUTs, count elements, and output the results.

distX(rng) or distY(rng) : Generates a random X or Y coordinate within the image bounds using the previously defined uniform distribution.

Calls the **DrawGaussian** function to add a Gaussian distribution centered at (X, Y) with standard deviations of 10 in both the X and Y directions in the ProccesIMG image.

Initializes the LUT array with a threshold of 200 and the flag B_Or_W set to 1

Draws a Gaussian distribution centered at the middle of the gaussian image with standard deviations of 10 in both the X and Y directions.

```
151 //part1
152 // Draw Gaussian distributions in random locations
153 for (int i = 0; i < 6; i++) {
154     X = distX(rng); // Generate a random X coordinate
155     Y = distY(rng); // Generate a random Y coordinate
156     DrawGaussian(ProccesIMG, X, Y, 10, 10);
157 }
158
159 // Save the processed image
160 StoreGrayImageAsGrayBmpFile(ProccesIMG, "Image241.bmp");
161 InitThresholdLUTpart1(LUT, 200, 1);
162 DrawGaussian(gaussian, int(NUMBER_OF_ROWS / 2), int(NUMBER_OF_COLUMNS / 2), 10, 10);
163 // Apply LUT on the Gaussian image
164 ImposeLUT(gaussian, LUT);
165 BPixelCountElem = ContElemInPicture(gaussian, MyROI);
166
167 ImposeLUT(ProccesIMG, LUT);
168
169 BPixelCount = ContElemInPicture(ProccesIMG, MyROI);
170
171 // Output the number of Gaussian elements found
172 cout << "Found " << int(BPixelCount / BPixelCountElem + 0.5) << " Gaussian elements." << endl;
```

24.6 – “main” function – part 3

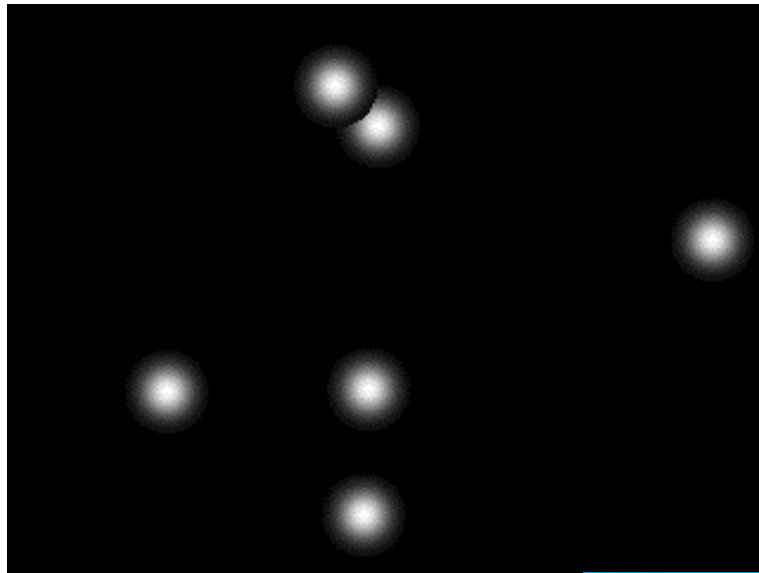
11

```
175 //part2
176 // Load a color image and convert to grayscale
177 LoadGrayImageFromTrueColorBmpFile(ProccesIMG, "Image242_color.bmp");
178 StoreGrayImageAsGrayBmpFile(ProccesIMG, "Image242_gray.bmp");
179 // Initialize LUT for thresholding the new image
180 InitThresholdLUT(LUT, 60, 65);
181
182 // Apply LUT on the processed image
183 ImposeLUT(ProccesIMG, LUT);
184
185 // Measure the size of elements in a specific ROI
186 MyROI.top = 127;
187 MyROI.bottom = 192;
188 MyROI.left = 9;
189 MyROI.right = 76;
190 BPixelCountElem = ContElemInPicture(ProccesIMG, MyROI);
191
192 // Count black pixels in the entire image
193 MyROI.top = 0;
194 MyROI.bottom = NUMBER_OF_ROWS;
195 MyROI.left = 0;
196 MyROI.right = NUMBER_OF_COLUMNS - 10;
197 BPixelCount = ContElemInPicture(ProccesIMG, MyROI);
198
199 // Save the final processed image
200 StoreGrayImageAsGrayBmpFile(ProccesIMG, "Image242.bmp");
201
202 // Output the number of elements found in the processed image
203 cout << "Found " << int(BPixelCount / BPixelCountElem + 0.5) << " elements in the processed image." << endl;
204
205 // Wait for user input before exiting
206 WaitForUserPressKey();
207 }
```

24.7 – created images and the counting results – **part 1**

12

Image241.bmp

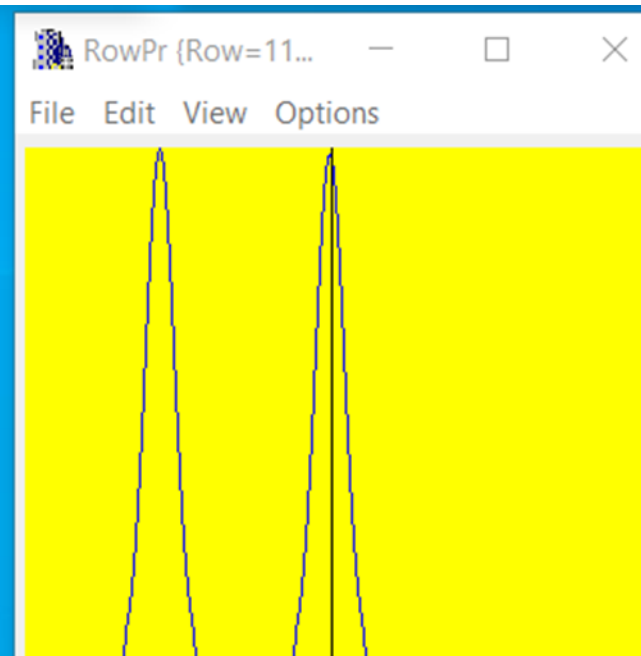
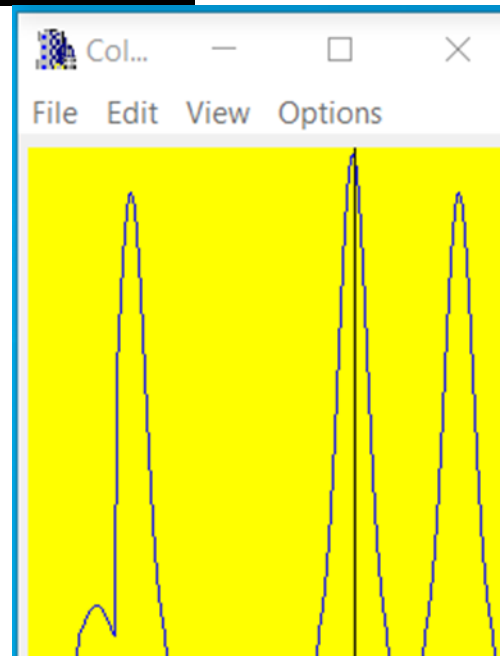


Found 6 Gaussian elements.

The printing of the program indicate that 6 gaussian elements were found!

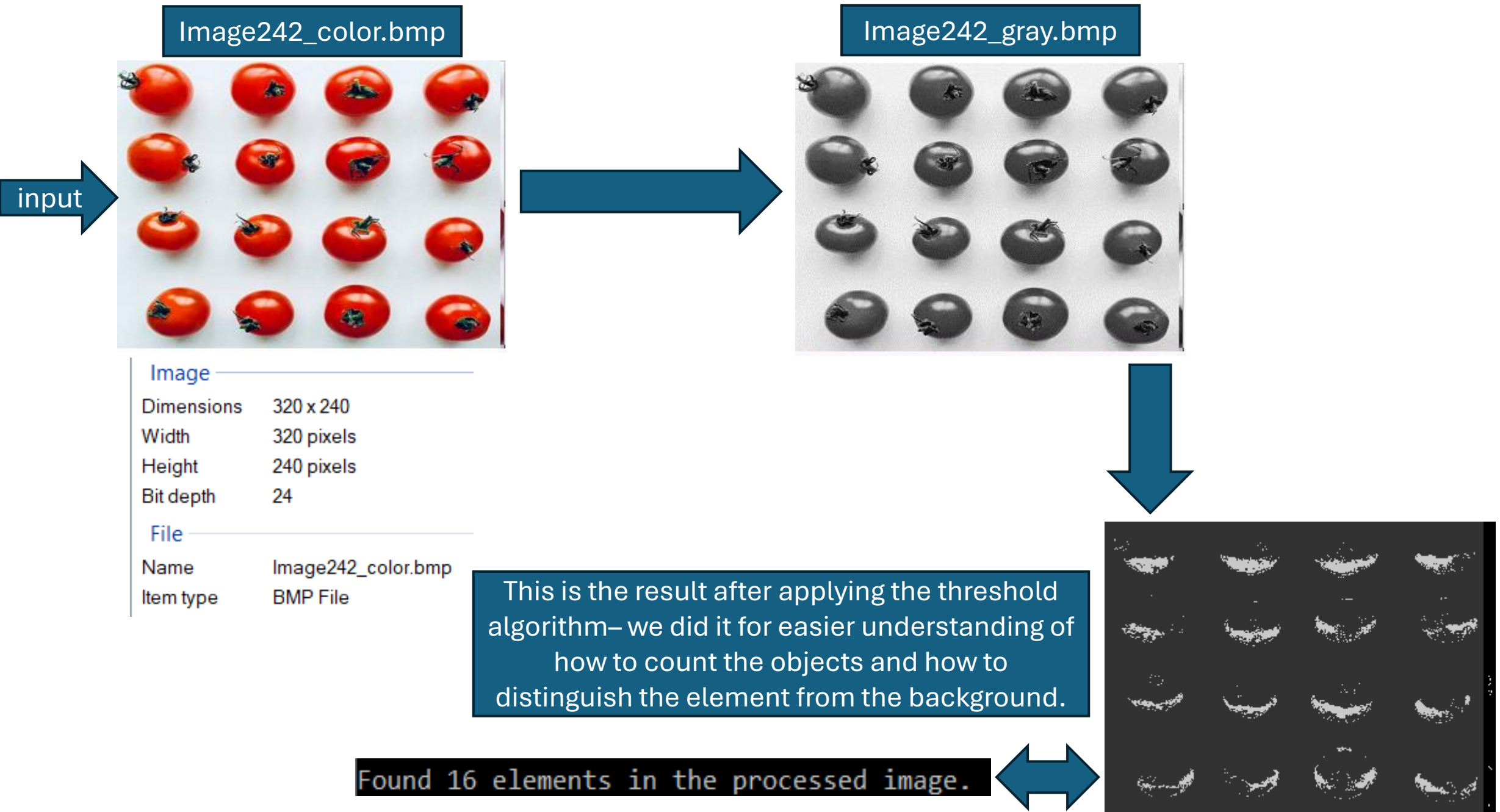
In addition – the profilers confirm that true gaussians were created.

The disruption in the column view (first from left) is due to overlapping between one gaussian and a small portion of other.



Point:	Mouse	Expl.	(M-E)
Row	76	76	0
Column	153	153	0
Red	250	250	0
Green	250	250	0
Blue	250	250	0

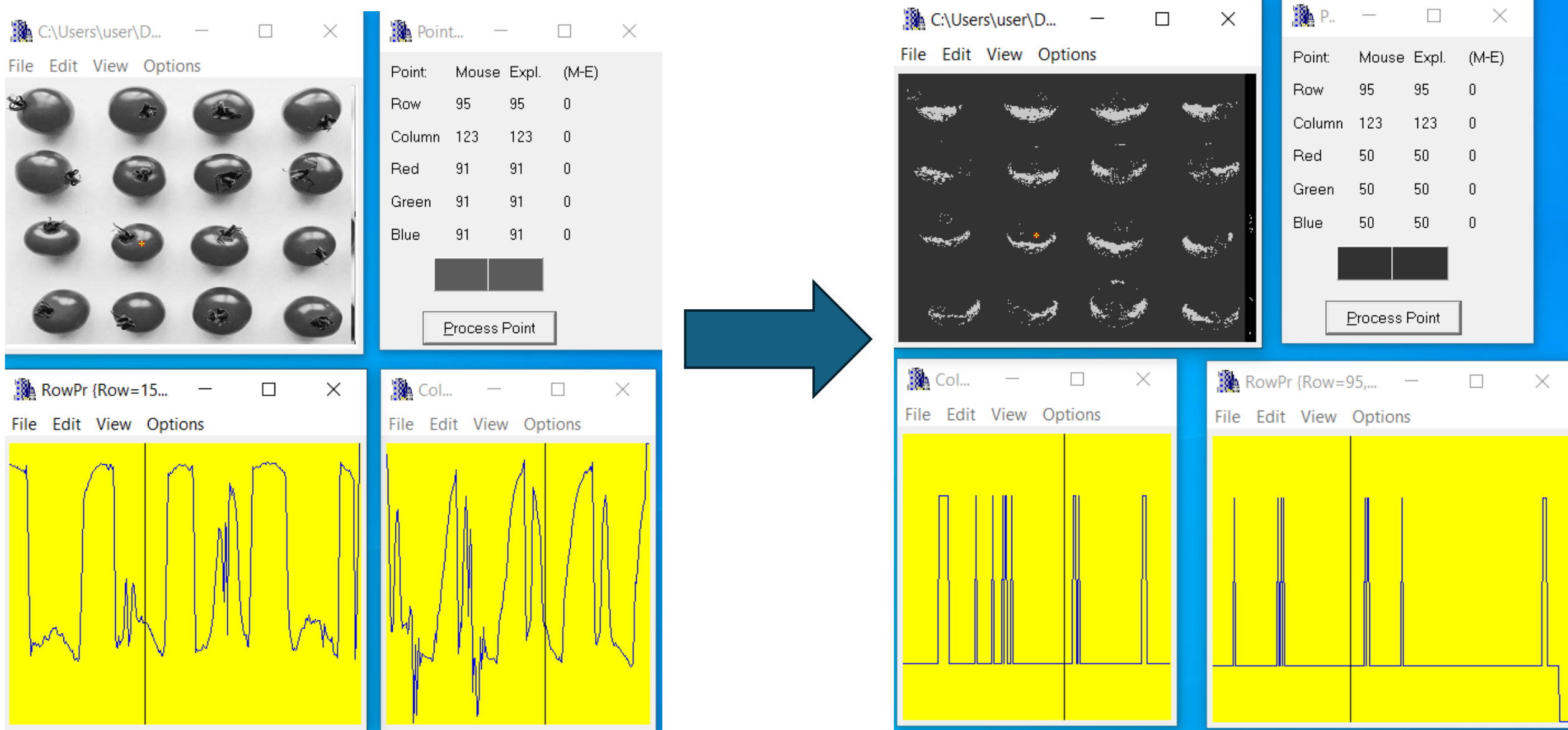
24.7 – created images and the counting results – **part 2**



24.7 – created images' profiles– part 2

14

We can see that the process we did converted the objects (tomatoes in that case) into impulse functions – they are much easier to count rather than the weird shapes before that.



24.8 – what did we learn?

1. We learned how to count similar objects with similar shape and color in a photo using binary threshold method (under known background)
2. We learned how to use uniform distribution for getting random values
3. We learned how to create random gaussians